

CS620T: Compilers - Principles and Implementation

Autumn 2026

Week 7: Local Optimizations (contd.)

Common subexpression elimination

- Goal: remove redundant computation, don't calculate the same expression multiple times

1: $A = B * C$

2: $E = B * C$

Keep the result of statement 1 in a temporary and reuse for statement 2

- Difficulty: how do we know when the same expression will produce the same result?

1: $A = B * C$

2: $B = \text{<new value>}$

3: $E = B * C$

B is “killed.” Any expression using B is no longer “available,” so we cannot reuse the result of statement 1 for statement 3

- This becomes harder with pointers (how do we know when B is killed?)

Common subexpression elimination

- Two varieties of common subexpression elimination (CSE)
- Local: within a single basic block → Maximal sequence of instructions that are executed one after another (i.e. there are no jump instructions OR no instruction is the target of a jump)
 - Easier problem to solve (why?)
- Global: within a single procedure or across the whole program
 - Intra- vs. inter-procedural
 - More powerful, but harder (why?)
 - Will come back to these sorts of “global” optimizations later

Local optimizations are done on basic blocks. Global optimizations on control flow graphs (CFGs), where the basic blocks are the nodes of the graph. Then, there are inter-procedural optimizations, which span function calls. Later on CFGs and other kinds of optimizations. 3

CSE in practice

- Idea: keep track of which expressions are “available” during the execution of a basic block
 - Which expressions have we already computed?
 - Issue: determining when an expression is no longer available
 - This happens when one of its components is assigned to, or “killed.”
- Idea: when we see an expression that is already available, rather than generating code, copy the temporary
 - Issue: determining when two expressions are the same

Maintaining available expressions

- For each 3AC operation in a basic block
 - Create name for expression (based on lexical representation)
 - If name not in available expression set, generate code, add it to set
 - Track register that holds result of and any variables used to compute expression
 - If name in available expression set, generate move instruction
 - If operation assigns to a variable, kill all dependent expressions

Example

3 Address Code	Available expression(s)	Killed expression(s)	Generated Code (assembly)
ADD A B T1	{}		ld a r1; ld b r2; add r1 r2 r1
ADD T1 C T2	{"A + B"}	ld c r5;	add r1 r5 r2
ADD A B T3	{"A + B", "T1 + C"}		mov r1 r3
ADD T1 T2 C	{"A + B", "T1 + C"}	{"T1+C"}	add r1 r2 r5 st r5 c
ADD T1 C T4	{"A + B", "T1 + T2"}	ld c r5;	add r1 r5 r4
ADD T3 T2 D	{"A + B", "T1 + T2", "T1 + C"}		add r3 r2 r6 st r6 d
	{"A + B", "T1 + T2", "T1 + C", "T3 + T2"}		

Downsides (CSE)

- What are some downsides to this approach? Consider the two highlighted operations

Three address code

```
+ A B T1
+ T1 C T2
+ A B T3
+ T1 T2 C
+ T1 C T4
+ T3 T2 D
```

Generated code

```
ADD A B R1
ADD R1 C R2
MOV R1 R3
ADD R1 R2 R5; ST R5 C
ADD R1 C R4
ST R5 D
```

T1 and T3 compute the same expression. This can be handled by an optimization called *value numbering*.

Aliasing

- One of the biggest problems in compiler analysis is to recognize aliases – different names for the same location in memory

exercise: are T1 and T3 aliased in previous example?

- Why do aliases occur?
 - Pointers referring to the same location
 - Function calls passing the same reference in two arguments
 - Arrays referencing the same element
 - Unions
- What problems does aliasing pose for CSE?
 - when talking about “live” and “killed” values in optimizations like CSE, we’re talking about particular variable names
 - In the presence of aliasing, we may not know which variables get killed when a location is written to

Memory disambiguation

- Most compiler analyses rely on *memory disambiguation*
 - Otherwise, they need to be too conservative and are not useful
- Memory disambiguation is the problem of determining whether two references point to the same memory location
 - *Points-to* and *alias* analyses try to solve this
 - Will cover basic pointer analyses in a later lecture

Single assignment form and its use in local optimizations

Single assignment form: a variable is assigned only once i.e. appears only once in LHS.

Aids CSE: $x = z + y$
...
 $x = z + y$

Neither z nor y can appear on the LHS here in single assignment form.

So, can be sure that this $z + y$ is the same expression as earlier. In the original code, if z or y were assigned to in between the two expressions, then we would have used different names, say, $z1 = ..$; $y1 = ..$; then the last expression would have to be rewritten as $x = z1 + y1$.

$x = z + y$
 $a = x$
 $x = 2 * x$

replace x with b



$b = z + y$
 $a = b$
 $x = 2 * b$

Aids copy propagation: can replace all the uses of a variable downstream

Aids dead code elimination: if the variable is never used later, can safely remove the statement where the variable is defined/assigned to.

Example – Local Optimizations

a=x**2

b=3

c=x

d=c*c

e=b*2

f=a+d

g=e*f

Example – Local Optimizations

- Algebraic simplification – exploiting mathematical properties of operators involved

a=x*x

b=3

c=x

d=c*c

e=b<<1

f=a+d

g=e*f

Example – Local Optimizations

- Copy and constant propagation

a=x*x		a=x*x
b=3		b=3
c=x		c=x
d=c*c	➔	d=x*x
e=b<<1		e=3<<1
f=a+d		f=a+d
g=e*f		g=e*f

Example – Local Optimizations

- Constant folding

```
a=x*x  
b=3  
c=x  
d=c*c  
e=b<<1  
f=a+d  
g=e*f
```



```
a=x*x  
b=3  
c=x  
d=x*x  
e=3<<1  
f=a+d  
g=e*f
```



```
a=x*x  
b=3  
c=x  
d=x*x  
e=6  
f=a+d  
g=e*f
```

Example – Local Optimizations

- CSE

a=x*x

b=3

c=x

d=x*x

e=6

f=a+d

g=e*f



a=x*x

b=3

c=x

d=a

e=6

f=a+d

g=e*f

Example – Local Optimizations

- Copy and Constant Propagation

a=x*x

b=3

c=x

d=a

e=6

f=a+d

g=e*f



a=x*x

b=3

c=x

d=a

e=6

f=a+a

g=6*f

Example – Local Optimizations

- Dead code elimination

a=x*x

b=3

c=x

d=a

e=6

f=a+a

g=6*f



a=x*x

f=a+a

g=6*f

Anything else?

a=x*x

f=2*a

g=6*f



a=x*x

~~f=2*a~~

g=12*a

Register Allocation

- Simple code generation (in CSE example): use a register for each temporary, load from a variable on each read, store to a variable at each write
- What are the problems?
 - Real machines have a limited number of registers – one register per temporary may be too many
 - Loading from and storing to variables on each use may produce a lot of redundant loads and stores

Register Allocation

- Goal: allocate temporaries and variables to registers to:
 - Use only as many registers as machine supports
 - Minimize loading and storing variables to memory (keep variables in registers when possible)
 - Minimize putting temporaries on stack (“spilling”)

Global vs. Local

- Same distinction as global vs. local CSE
 - Local register allocation is for a single basic block
 - Global register allocation is for an entire function

Does inter-procedural register allocation make sense? Why? Why not?

Hint: think about caller-save, callee-save registers

When we handle function calls, registers are pushed/popped from stack

Top-down register allocation

- For each basic block
 - Find the number of references of each variable
 - Assign registers to variables with the most references
- Details
 - Keep some registers free for operations on unassigned variables and spilling
 - Store *dirty* registers at the end of BB (i.e., registers which have variables assigned to them)
 - Do not need to do this for temporaries (why?)

Bottom-up register allocation

- Smarter approach:
 - Free registers once the data in them isn't used anymore
- Requires calculating *liveness*
 - A variable is live if it has a value that *may* be used in the future
- Easy to calculate if you have a single basic block:
 - Start at end of block, all local variables marked dead
 - If you have multiple basic blocks, all local variables defined in the block should be *live* (they may be used in the future)
 - When a variable is used, mark as live, record use
 - When a variable is defined, record def, variable dead above this
 - Creates chains linking uses of variables to where they were defined
- We will discuss how to calculate this across BBs later

Bottom-up register allocation

For each tuple $op\ A\ B\ C$ in a BB, do

$R_x = \text{ensure}(A)$

$R_y = \text{ensure}(B)$

if A dead after this tuple, $\text{free}(R_x)$

if B dead after this tuple, $\text{free}(R_y)$

$R_z = \text{allocate}(C)$ //could use R_x or R_y

generate code for op

mark R_z dirty

At end of BB, for each dirty register

generate code to store register into appropriate variable

- We will present this as if A, B, C are variables in memory. Can be modified to assume that A, B and C are in virtual registers, instead

Bottom-up register allocation

`ensure(opr)`

if opr is already in register r

return r

else

r = allocate(opr)

generate load from opr into r

return r

Liveness Example

- What is live in this code? *Recall: a variable is live only if its value is used in future.*

	Live	Comments
1: A = B + C	{A, B}	Used B, C Killed A
2: C = A + B	{A, B, C}	Used A, B Killed C
3: T1 = B + C	{A, B, C, T1}	Used B, C Killed T1
4: T2 = T1 + C	{A, B, C, T2}	Used T1, C Killed T2
5: D = T2	{A, B, C, D}	Used T2, Killed D
6: E = A + B	{C, D, E}	Used A, B Killed E
7: B = E + D	{B, C, D}	Used E, D Killed B
8: A = C + D	{A, B}	Used C, D Killed A
9: T3 = A + B	{T3}	Used A, B Killed T3
10: WRITE(T3)	{}	Used T3

Bottom-up register allocation - Example

		Registers			
	Live	R1	R2	R3	R4
1: A = 7	{A}	A*			mov 7 r1
2: B = A + 2	{A, B}	A*	B*		add r1 2 r2
3: C = A + B	{A, B, C}	A*	B*	C*	add r1 r2 r3
4: D = A + B	{B, C, D}	D*	B*	C*	add r1 r2 r1 (free r1 - dead)
5: A = C + B	{A, B, C, D}				
6: B = C + B	{A, B, C, D}				
7: E = C + D	{A, B, C, D, E}				
8: F = C + D	{A, B, E, F}				
9: G = A + B	{E, F, G}				
10: H = E + F	{H, G}				
11: I = H + G	{I}				
12: WRITE(I)	{}				